

0.1. Оптимізація геометрії

Геометрія молекули є фундаментальною характеристикою, від якої залежить практично весь спектр її фізико-хімічних властивостей: енергія, електронна структура, спектри коливань і збуджень, поляризованість, багаточастинні відгуки та реакційна здатність. У межах наближення Борна—Оппенгеймера ця геометрія визначає форму поверхні потенціальної енергії, а отже формує локальну поведінку електронної хмари, умови зв'язування та всі похідні величини.

Будь-яке відхилення від рівноважної структури — навіть незначне — неминуче викликає систематичні спотворення. Коливальні частоти зсуваються через некоректно визначений гесіан; дипольні, квадрупольні та вищі моменти набувають залежності від деформації, яка не має фізичного сенсу; енергії вертикальних переходів зміщуються, а похідні, що лежать в основі теорії відгуку, стають некоректними, оскільки розраховані не в точці мінімуму, де вони математично визначені.

Таким чином, оптимізація геометрії не є технічним попереднім кроком, а виступає фундаментальною частиною квантово-хімічного моделювання. Вона забезпечує коректність усіх наступних розрахунків, дозволяє строго застосовувати варіаційні та аналітичні методи, та гарантує, що подальший аналіз ґрунтуватиметься на фізично осмисленій, енергетично виправданій структурі.

Для будь-якої квантово-хімічної моделі енергія $E(\mathbf{R})$ визначається положеннями ядер, і завдання оптимізації полягає в тому, щоб знайти таку конфігурацію

$$\mathbf{R}_0 = \arg \min_{\mathbf{R}} E(\mathbf{R}),$$

яка відповідає стаціонарній точці поверхні потенційної енергії. У межах наближення Борна—Оппенгеймера ця точка інтерпретується як рівноважна структура, де сили повністю компенсуються:

$$\nabla_{\mathbf{R}} E(\mathbf{R}_0) = \mathbf{0}.$$

Гradient $\nabla_{\mathbf{R}} E$ дає напрямок найшвидшого спаду енергії, але не враховує геометрію поверхні потенціальної енергії. Тому використання лише градієнта (градієнтний спуск) неефективне. Гесіан

$$H_{\alpha\beta} = \frac{\partial^2 E}{\partial R_{\alpha} \partial R_{\beta}}$$

містить інформацію про локальну кривизну, тобто про те, як змінюється градієнт. Наявність хорошої оцінки гесіана дозволяє робити «Ньютонівські» кроки, які набагато швидше приводять до мінімуму. Саме тому сучасні оптимізатори експлуатують наближення до гесіана й оновлюють його в міру накопичення даних.

0.1.1. Оптимізатори геометрії в PySCF

Документацію PySCF з оптимізації можна знайти за посиланням [Geometry optimization](#).

PySCF надає два інструменти для оптимізації геометрії: застарілий [PyBerny](#)-алгоритм та сучасний оптимізатор **geomeTRIC**. Перший підтримується для сумісності з попередніми версіями й історичного використання, та через інтерфейси до [geomeTRIC](#), який є рекомендованим методом для всіх актуальних задач оптимізації, включаючи великі системи, погані початкові геометрії та розрахунки з обмеженнями.

Встановлення **PyBerny** виконується командою:

```
pip install berny
```

Використання **PyBerny**:

```
from pyscf import gto, scf
mol = gto.M(atom='N 0 0 0; N 0 0 1.2', basis='ccpvdz')
mf = scf.RHF(mol)

from pyscf.geomopt.berny_solver import optimize
mol_eq = optimize(mf, maxsteps=100)
print(mol_eq.tostring())
```

Встановлення **geomeTRIC** виконується командою:

```
pip install geometric
```

Використання **geomeTRIC**:

```
from pyscf import gto, scf
mol = gto.M(atom='N 0 0 0; N 0 0 1.2', basis='ccpvdz')
mf = scf.RHF(mol)

# geomeTRIC
from pyscf.geomopt.geometric_solver import optimize
mol_eq = optimize(mf, maxsteps=100)
print(mol_eq.tostring())
```

В PySCF є ще один спосіб по оптимізації — це створення `optimizer()` з класу `Gradients`:

```
# geomeTRIC
mol_eq = mf.Gradients().optimizer(solver='geomeTRIC').kernel()
print(mol_eq.tostring())

# PyBerny
mol_eq = mf.Gradients().optimizer(solver='berny').kernel()
print(mol_eq.tostring())
```

Для бекенду `geomeTRIC` критерії збіжності контролюються такими параметрами:

```
conv_params = { # Це налаштування за замовчуванням
    'convergence_energy': 1e-6, # Eh
    'convergence_grms': 3e-4, # Eh/Bohr
    'convergence_gmax': 4.5e-4, # Eh/Bohr
    'convergence_drms': 1.2e-3, # Angstrom
    'convergence_dmax': 1.8e-3, # Angstrom
}
mol_eq = optimize(mf, **conv_params)
mol_eq = mf.Gradients().optimizer(solver='geomeTRIC').kernel(conv_params)
```

Для бекенду `PyBerny` критерії збіжності контролюються такими параметрами:

```
conv_params = { # Це налаштування за замовчуванням
    'gradientmax': 0.45e-3, # Eh/[Bohr|rad]
    'gradientrms': 0.15e-3, # Eh/[Bohr|rad]
    'stepmax': 1.8e-3, # [Bohr|rad]
    'steprms': 1.2e-3, # [Bohr|rad]
}
mol_eq = optimize(mf, **conv_params)
mol_eq = mf.Gradients().optimizer(solver='berny').kernel(conv_params)
```

0.1.2. Обмеження (Constraints)

У практичних квантово-хімічних розрахунках оптимізація геометрії не завжди повинна проходити у повністю вільному просторі $3N$ декартових координат. Існує широкий клас задач, у яких частина внутрішніх координат повинна бути зафіксована або змінюватися у контрольований спосіб. Такі *обмеження* (*constraints*) дозволяють включити додаткову фізичну інформацію або реалізувати специфічні вимоги до структури.

`geomeTRIC` підтримує користувацькі обмеження. Обмеження можна вказати у текстовому файлі. Його формат описано в [онлайн-документації](#) або в [шаблонному файлі](#). Потім ім'я файлу з обмеженнями потрібно передати PySCF:

```
params = {"constraints": "constraints.txt",}
mol_eq = optimize(mf, **params)
mol_eq = mf.Gradients().optimizer(solver='geomeTRIC').kernel(params)
```

Файл обмежень `constraints.txt` — це текстовий файл, де кожен рядок задає одну інструкцію.

Основні типи обмежень

- **distance** — фіксація між'ядерної відстані;
- **angle** — фіксація валентного кута;
- **dihedral** — фіксація торсійного кута;
- **freeze atom** — заморожування координат атома;
- **freeze --- nslation/rotation** — фіксація глобальних мод;
- **lineq** — лінійне рівняння на координати;
- **group** — заморожування групи атомів як єдиного фрагмента;
- **scan** — параметричний перегляд координати (напівоптимізація).

Приклади файлу `cons --- ints.txt`:

```
# Фіксація довжини зв'язку O-H (атоми 1 і 2)
distance 1 2 0.960

# Фіксація валентного кута H-O-H
angle 1 2 3 104.5

# Фіксація торсійного кута C-C-O-H
dihedral 5 6 7 8 180.0

# Заморожування атома №5 (всі 3 координати)
freeze atom 5

# Заморожування двох атомів як жорсткої групи
group freeze 10 11 12

# Заморожування глобальних мод (інколи потрібно для кластерів)
freeze rotation
freeze --- nslation

# Примусове накладання планарності через лінійне рівняння
#  $x_3 - x_4 = 0$ 
lineq 1*x3 -1*x4 = 0.0

# Примус загальної симетрії: атоми 2 і 3 на однаковій висоті
lineq 1*z2 -1*z3 = 0.0

# Дзеркальна симетрія: середня точка між 1-2 лежить у площині XY
lineq 0.5*z1 +0.5*z2 = 0.0

# Параметричне сканування відстані 1-2 (використовується у scan jobs)
scan distance 1 2 0.90 1.50 0.05

# Сканування торсії з кроком 10 градусів
scan dihedral 3 4 5 6 0.0 360.0 10.0
```

Усередині PySCF файл безпосередньо передається в `geometric.optimize`, і `geomeTRIC` інтерпретує всі інструкції.

0.2. Збуджені стани

Оптимізація геометрії збуджених електронних станів є необхідною для аналізу спектроскопії, розрахунку флуоресценції, визначення безвипромі-

новальних переходів та пошуку міжсистемних перетинів. На відміну від основного стану, поверхня потенційної енергії збудженого стану істотно відрізняється за топологією, що змінює рівноважні довжини зв'язків, кути та торсії:

$$\mathbf{R}_0^{(S_n)} = \arg \min_{\mathbf{R}} E_{S_n}(\mathbf{R}).$$

Для коректного пошуку стаціонарної точки треба явно вказати, *який саме електронний стан* оптимізується, оскільки градієнт є стан-специфічним. У PySCF це реалізовано через об'єкти `Gradients`, що отримують інформацію про цільовий стан з електронного розрахунку (TDDFT, CIS, CASSCF або MCSCF).

Для оптимізації геометрії збуджених станів потрібно вказати стан, який оптимізується, у відповідних об'єктах `Gradients`:

```
'''
Оптимізація геометрії збуджених станів

Примітка: при оптимізації збуджених станів стани можуть перевертатися, що
може спричинити проблеми зі збіжністю в оптимізаторі геометрії.
'''

from pyscf import gto
from pyscf import scf
from pyscf import ci, tdscf, mcscf
from pyscf import geomopt

mol = gto.Mole()
mol.atom="N; N 1, 1.1"
mol.basis= "6-31g"
mol.build()
mol1 = mol.copy()

mf = scf.RHF(mol).run()

mc = mcscf.CASCI(mf, 4,4)
mc.fcisolver.nstates = 3
excited_grad = mc.nuc_grad_method().as_scanner(state=2)
mol1 = excited_grad.optimizer().kernel()
# or
#mol1 = geomopt.optimize(excited_grad)

td = tdscf.TDHF(mf)
td.nstates = 5
excited_grad = td.nuc_grad_method().as_scanner(state=4)
mol1 = excited_grad.optimizer().kernel()
# or
#mol1 = geomopt.optimize(excited_grad)

myci = ci.CISD(mf)
myci.nstates = 2
excited_grad = myci.nuc_grad_method().as_scanner(state=1)
mol1 = excited_grad.optimizer().kernel()
# or
#geomopt.optimize(excited_grad)
```

Для прикладів оптимізації геометрії стан-специфічного та усередненого по станах CASSCF див. приклад `c 12-mcscf_excited_states.py`.

0.3. Callback під час оптимізації

Під час оптимізації геометрії важливо мати доступ до проміжної інформації кожного кроку: енергії, градієнтів, зарядів, молекулярних орбіталей та інших характеристик. Стандартні оптимізатори надають лише фінальний результат, що ускладнює діагностику збіжності та аналіз поведінки системи. Механізм **callback** дозволяє виконувати довільний код після кожного кроку оптимізації, забезпечуючи гнучке керування процесом, відстеження стану молекули, запис проміжних структур і розрахунків додаткових властивостей. Це робить callback незамінним інструментом у складних розрахунках, зокрема в QM/MM, оптимізації перехідних та збуджених станів, а також під час збору даних для власних методів аналізу чи машинного навчання.

Функції callback можна викликати на кожному кроці оптимізації. Наступний приклад показує, як додати аналіз заряду для кожної геометрії під час оптимізації.

```
#!/usr/bin/env python

'''
Оптимізація молекулярної геометрії в середовищі зарядів QM/MM.
'''

from pyscf import gto, scf
from pyscf.geomopt import berny_solver
from pyscf.geomopt import geometric_solver

mol = gto.M(atom='''
C      0.000000    0.000000    -0.542500
O      0.000000    0.000000     0.677500
H      0.000000    0.9353074360871938  -1.082500
H      0.000000   -0.9353074360871938  -1.082500
''',
            basis='3-21g')

mf = scf.RHF(mol)

# Запустити функцію analyze в callback
def cb(envs):
    mf = envs['g_scanner'].base
    mf.analyze(verbose=4)

#
# Метод 1: Передати callback до функції optimize
#
geometric_solver.optimize(mf, callback=cb)

berny_solver.optimize(mf, callback=cb)

#
# Метод 2: Додати callback до оптимізатора геометрії
#
```

```
opt = mf.nuc_grad_method().as_scanner().optimizer()
opt.callback = cb
opt.kernel()
```

0.4. Початковий гесіан в оптимізації геометрії

Гесіан (матриця других похідних енергії за ядерними координатами) відіграє ключову роль у методах другого порядку та квазі-ньютонівських алгоритмах. Він визначає напрямок і довжину кроку, прогнозує зміну градієнта та дозволяє ідентифікувати характер стаціонарної точки (мінімум чи сідлова точка).

Сучасний оптимізатор **geomeTRIC**, який є рекомендованим у **PySCF**, працює у делокалізованих внутрішніх координатах і за замовчуванням генерує евристичний початковий гесіан на основі простої моделі силового поля валентних зв'язків. Цього достатньо для переважної більшості задач оптимізації основного стану середнього розміру.

Проте в низці випадків якість початкового наближення гесіана суттєво впливає на швидкість і надійність збіжності:

- пошук перехідних станів (потрібне правильне негативне власне значення);
- великі та гнучкі системи (>50–100 атомів),
- сильно деформовані або штучні початкові геометрії.

У **PySCF** аналітичний гесіан доступний для більшості методів (HF, DFT, MP2, CCSD, CASSCF тощо) і обчислюється так:

```
hess = mf.Hessian().kernel() # shape (natm, 3, natm, 3)
H = hess.transpose(0, 2, 1, 3).reshape(-1, -1) # 3N × 3N декартовий гесіан
```

Отриману матрицю можна зберегти та повторно використати:

```
import numpy as np
np.save("initial_hessian.npy", H) # збереження
H = np.load("initial_hessian.npy") # відновлення
```

Передача користувацького гесіана в **geomeTRIC** надзвичайно проста:

```
from pyscf.geomopt.geometric_solver import optimize

mol_eq = optimize(mf, hessian=H)
# або через інтерфейс Gradients
mol_eq = mf.Gradients().optimizer(solver='geomeTRIC').kernel(hessian=H)
```

При цьому **geomeTRIC** автоматично проєктує декартовий гесіан на свої внутрішні координати, відсікає трансляційно-обертальні моди й використовує його як стартове наближення. Подальше оновлення відбувається стандартними квазі-ньютонівськими схемами (зазвичай L-BFGS).

0.4.1. Практичні рекомендації

- Для звичайних молекул (до ≈ 50 атомів) і розумних початкових геометрій передавати гесіан не потрібно — евристика **geomeTRIC** працює відмінно.
- Для пошуку TS, великих систем або «холодного старту» з поганої геометрії — завжди обчислюйте і передавайте точний гесіан.
- У релаксаційному скануванні чи IRC — зберігайте гесіан з попередньої точки й передавайте як **hessian=**. Це часто скорочує кількість кроків у 2–5 разів.
- Якщо аналітичний гесіан недоступний, можна використати числовий гесіан з методу `mf.Hessian().set(numerical=True)`.

Таким чином, хоча **geomeTRIC** і не вимагає від користувача думати про гесіан, можливість передати точну аналітичну матрицю других похідних залишається потужним інструментом для розв’язання складних і великих задач оптимізації.